Comparative Evaluation of Inverse Kinematics Methods for Pick-and-Place with HEAL and Franka Panda

ME639 – Introduction to Robotics | Lab 3: Pick and Place of Objects | IIT Gandhinagar
Kshitij Suresh Giri (23110177) | 16th October 2026

Video submission: https://youtu.be/yv1NYxJ2LtA
Github: https://github.com/Kshitij-Giri/me639-itr-github/tree/main/Lab.

1. Introduction and Goals

Inverse kinematics (IK) is central to robotic manipulation: it determines the joint motions required to achieve a desired end-effector pose. This work studies IK through a complete pick-and-place task in MuJoCo [1], using two manipulators: the six-joint HEAL arm with a Robotiq 2F-85 gripper, and the seven-joint Franka Panda with its parallel-jaw hand [2]. Both robots must grasp a 40 mm cube from a table, lift and transport it, and place it inside an open container. The primary objective is to implement and compare three IK approaches—off-the-shelf Mink [3], custom damped least-squares (DLS) [4, 5], and custom quadratic programming (QP) [7, 8]—within this common task.

Goals. (i) Implement the complete pick-and-place sequence, including approach, grasp, lift, transfer, release, and placement verification, for both robots. (ii) Compare Mink, DLS, and QP under identical initial object poses and reproducible episode seeds, using success rate, solve time, obstacle clearance, and joint-limit and velocity-constraint violations where available. (iii) Establish feasible object-placement regions through task-specific workspace analysis and optimize a separate table for each robot. This supports the IK evaluation by ensuring that randomized cube and container configurations are suitable for the intended manipulation task. (iv) Reduce collision risk during grasping and transfer through feasible placement sampling, sufficient cube-to-container separation, and appropriate motion and joint constraints.

Contributions. (1) A complete pick-and-place implementation for HEAL and Franka that can use three independent, generalized IK solvers through robot-specific task scripts. (2) A workspace-driven table-design procedure that considers grasp feasibility and complete pick–transfer–place motions, rather than relying on raw reachability alone. (3) A reproducible experiment runner (`run_experiments.py`) configured for 150 episodes (2 robots × 3 IK methods × 25 seeds), with identical initial layouts across methods for paired comparisons. It records episode outcomes and available performance metrics for quantitative analysis.

2. Methods

Environment and workspace. The pick-and-place task was implemented in MuJoCo for the HEAL and Franka Panda arms, using a cube and a tray as the target. Workspace sampling guided table design. For HEAL, downward-facing TCP poses were tested using Mink IK to identify a large reachable rectangular region at a common height. For Franka, feasibility was progressively evaluated for grasping, placement, collision-aware waypoints, and sampled pick–place transfers. The resulting tables were:

Robot	Table ($L \times W$)	Top height	Centre (x, y)
HEAL	0.45×0.82 m	0.20 m	$(0.345, 0)$ m
Franka Panda	0.50×1.00 m	0.10 m	$(0.45, 0)$ m

Both tables have thickness 0.04 m. These dimensions are based on sampled workspace feasibility and do not guarantee that every continuous trajectory is collision-free.

Randomization and collision avoidance. Cube and tray positions and yaw were randomized within the tabletop bounds, ensuring that both objects remained on the table. The fully open gripper width defined a minimum separation threshold between the cube and tray, with a tolerance margin; invalid layouts were rejected. This clearance constraint reduces the risk of gripper–container collisions during grasping by ensuring sufficient separation between the randomized cube and tray positions. During execution, the cube was grasped and lifted before transfer to the tray, followed by a controlled descent and release. The lift–transfer–descent sequence helps reduce the risk of gripper–tray collisions during transfer. These measures reduce specific collision risks but do not guarantee collision-free motion throughout the entire trajectory.

Inverse kinematics. All methods used closed-loop differential IK based on TCP pose error and the arm Jacobian, with the same target, gain, timestep, and joint-speed cap:

- **Mink:** off-the-shelf task-based IK, used as the baseline.
- **Damped least squares (DLS):** $\dot{q} = (J^\top J + \lambda I)^{-1} J^\top \dot{x}_d$, with $\lambda = 10^{-3}$ for regularization near singularities.
- **Quadratic programming (QP):** minimizes the same regularized tracking objective as DLS, subject to joint-position and joint-velocity bounds.

Solver failures temporarily held the arm still; an episode was aborted after 50 consecutive IK failures.

Experimental protocol. Each robot–method combination was evaluated over 25 seeded episodes (150 episodes total). Success required the cube to be grasped and placed inside the tray without being dropped. Success, failure stage and reason, IK computation time, minimum clearance, solver diagnostics, and joint-position/velocity-limit violations were logged.

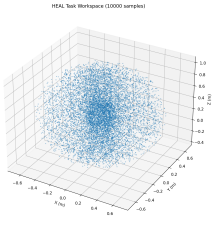
Methods were compared using identical per-robot seeds, summary metrics, plots, and paired exact McNemar tests on success outcomes. Failure logs and states were retained for analysis.

3. Results

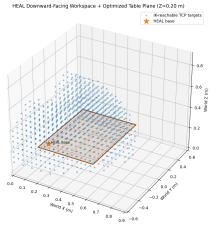
3.1 Workspace and table design

Workspace analysis was used to select tabletop regions where the robots could perform the pick-and-place task. Figures 1 and 2 show four complementary views for each robot. The task-workspace plot shows the sampled positions considered for the task; the 3D dexterous-workspace plot shows reachable end-effector targets; the height-optimization plot compares feasible tabletop areas at different heights; and the table-plane plot shows the selected tabletop region in the horizontal plane.

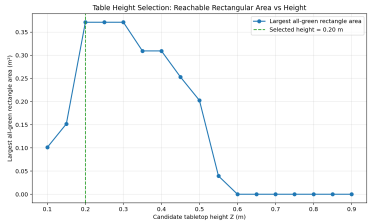
For HEAL, the selected tabletop height was 0.20 m, with a sampled rectangular region of approximately 0.45×0.825 m (area 0.371 m^2). Its dexterous-workspace analysis used sampled IK reachability and did not perform full robot/object collision validation. For Franka Panda, the selected height was 0.10 m, giving a table of 0.50×1.00 m (area 0.50 m^2). At this height, all 171 sampled cube cells and all 171 container cells passed L2 validation; all 40 sampled pick–transfer–place pairs passed L3, and all four physics replays succeeded. These checks provide evidence for the selected table design, but do not prove collision-free motion at every point in continuous space.



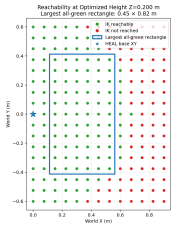
(a) Task workspace.



(b) 3D dexterous workspace.

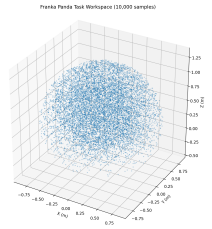


(c) Height optimization.

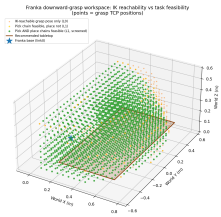


(d) Selected tabletop region.

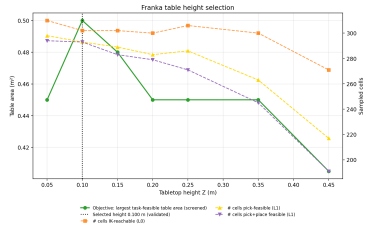
Figure 1: HEAL workspace analysis. The plots show the task sampling, sampled 3D reachability, height-dependent workspace analysis, and the final selected tabletop region.



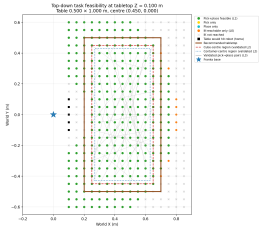
(a) Task workspace.



(b) 3D dexterous workspace.



(c) Height optimization.



(d) Selected tabletop region.

Figure 2: Franka Panda workspace analysis. The height-optimization plot supports the choice of the 0.10 m tabletop height, and the table-plane plot shows the selected region.

3.2 Experimental protocol

The comparison comprised 150 episodes: 25 for each robot–method combination. For each robot, identical seeds produced identical initial layouts across all three IK methods, and all 150 episodes matched their robot-specific reference layouts. Success rates are reported with 95% Wilson confidence intervals; paired method outcomes were

compared using exact McNemar tests. The logged metrics include IK computation time, total episode time, solver calls, minimum signed clearance, and joint-limit and velocity violations.

3.3 Quantitative comparison

Table 1 compares the performance of Mink, damped least squares (DLS), and quadratic programming (QP) over 25 episodes for each robot–method combination. All three methods completed 25/25 HEAL episodes. For Franka Panda, Mink and QP also achieved 25/25 successes, whereas DLS achieved 23/25 (92%). Both Franka–DLS failures occurred during descent and were recorded as motion timeouts; their analysis is presented in Section 4.

Table 1: Experimental results over 25 episodes per robot–method combination. Time values are mean ± standard deviation.

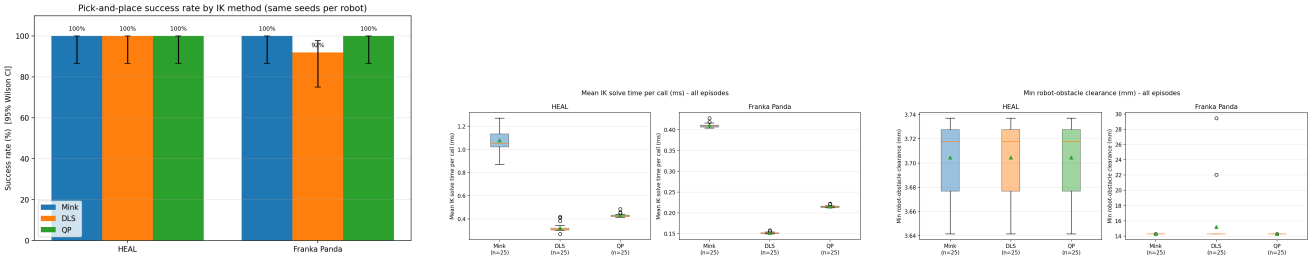
Robot	Method	Success (95% CI)	IK time/episode (s)	Total episode (s)	IK time/call (ms)	Calls/episode	Min. clearance (mm)	P/V episodes
HEAL	Mink	25/25 (86.7–100%)	1.822 ± 0.390	18.78 ± 4.11	1.080	1677	3.7	0/7
	DLS	25/25 (86.7–100%)	0.539 ± 0.107	16.86 ± 3.09	0.321	1677	3.7	0/7
	QP	25/25 (86.7–100%)	0.720 ± 0.119	16.34 ± 2.62	0.429	1677	3.7	0/7
Franka	Mink	25/25 (86.7–100%)	0.765 ± 0.134	5.38 ± 0.77	0.410	1868	14.3	0/0
	DLS	23/25 (75.0–97.8%)	0.283 ± 0.049	4.81 ± 0.76	0.152	1867	15.2	2/0
	QP	25/25 (86.7–100%)	0.401 ± 0.069	5.01 ± 0.71	0.215	1868	14.3	0/0

The table reports the following measures: **Success** is the number of episodes in which the cube was grasped and placed inside the tray; the 95% confidence interval (CI) indicates uncertainty in the estimated success rate. **IK time/episode** is the total time spent computing inverse kinematics (IK) during one episode, while **total episode time** includes IK computation, simulation, control, and task execution. **IK time/call** is the average computation time for one IK update, and **calls/episode** is the number of IK updates made during an episode. **Minimum clearance** is the mean of the smallest signed distance recorded in each episode between robot geometry and the table, tray, or floor. Finally, **P/V episodes** counts episodes with at least one joint-position violation / measured joint-speed violation.

DLS was the fastest method computationally, with mean IK times per call of 0.321 ms for HEAL and 0.152 ms for Franka. However, lower computation time did not guarantee successful task completion: the two Franka–DLS descent timeouts reduced its success rate to 92%. The mean total episode times were also different from the IK computation times because they include the full simulated task execution.

The exact paired McNemar tests found no statistically significant difference in success outcomes between methods: all HEAL pairwise comparisons gave $p = 1.00$; for Franka, Mink–DLS and DLS–QP gave $p = 0.50$, while Mink–QP gave $p = 1.00$. These results should be interpreted cautiously because only two paired outcomes differed in the Franka comparisons involving DLS.

The mean per-episode minimum clearance was approximately 3.7 mm for HEAL and 14.3 mm for Franka with Mink/QP, compared with 15.2 mm for Franka with DLS. No commanded joint-speed violations were recorded. Measured joint-speed violations occurred in seven episodes for each HEAL method and none for Franka. Joint-position violations occurred in two Franka–DLS episodes.



(a) Success rate. **(b)** IK time per call. **(c)** Minimum clearance.

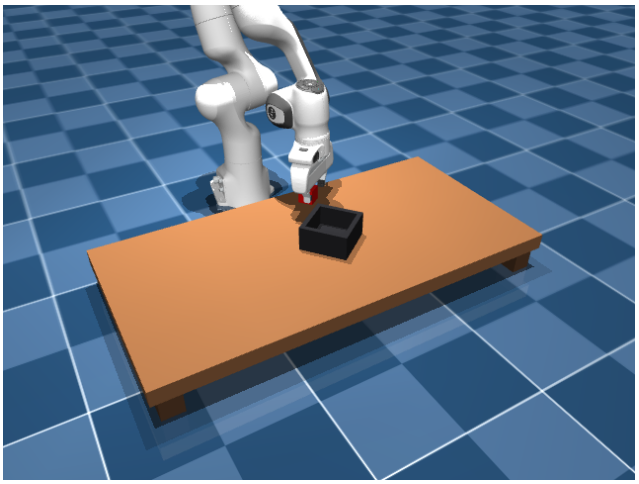
Figure 3: Comparison of Mink, DLS and QP using identical episode seeds. Success-rate bars show 95% Wilson confidence intervals.

4. Failure Analysis and Mitigations

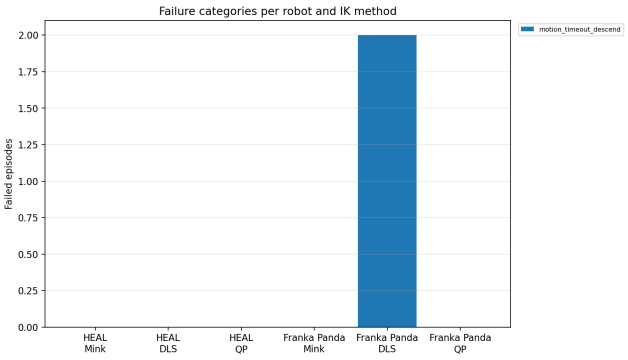
Of the 150 episodes, two failed. Both were Franka–DLS descent timeouts (Stage 2), at seeds 316063302 and 1471704793. No failures were recorded for HEAL, Franka–Mink, or Franka–QP.

Table 2: Observed failures and evidence-based mitigations.

Robot / IK	Count	Stage	Observed evidence and mitigation
Franka / DLS	2	Descent	Motion timed out before reaching the target. In a recorded example, position error was 30.1 mm (4 mm tolerance), with 2.77° tilt error and 0.03° yaw error. Check reachability and joint-limit behaviour; consider reducing descent speed or relaxing the target tolerance if task requirements allow.



(a) Franka–DLS failure, seed 316063302.



(b) Recorded failures by robot and method.

Figure 4: Failure evidence from the experiment run.

Interpretation. All initial layouts matched their reference layouts, and both failures occurred during descent rather than from invalid initial placements. The logs show two Franka–DLS episodes with joint-position-limit violations; DLS does not impose hard joint-position bounds. These observations make joint limits and target tracking worth investigating, but do not establish a single cause for the timeouts. No mitigation rerun was performed, so the proposed changes remain recommendations rather than verified fixes.

5. Discussion and Conclusions

Comparison of methods. DLS had the lowest mean IK computation time per call for both robots (0.321 ms for HEAL and 0.152 ms for Franka). All methods achieved 25/25 successes with HEAL; for Franka, Mink and QP achieved 25/25, while DLS achieved 23/25. The paired McNemar tests found no statistically significant differences, but the small number of failures limits the conclusions that can be drawn. Faster IK computation therefore did not guarantee the highest task success rate.

Key findings. Workspace-based table selection enabled successful execution in all tested HEAL episodes and all Franka Mink/QP episodes. The randomized layouts were consistent across methods. The two Franka–DLS failures and its joint-position violations indicate that joint-limit handling and descent tracking deserve further investigation.

Limitations and future work. Workspace feasibility was evaluated at sampled points and waypoints, and grasping was tested only in simulation. Only 25 seeds were tested per robot–method combination. Future work should investigate joint-limit-aware DLS, adaptive descent speeds, and alternative target poses, then rerun the same seeded tests to verify whether these changes improve success.

Conclusion. DLS was computationally fastest, while Mink and QP completed all tested episodes for both robots; the two Franka–DLS descent timeouts highlight the importance of evaluating task success alongside IK computation time.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ IROS*, 2012, pp. 5026–5033. https://doi.org/10.1109/IROS.2012.6386109

[2] K. Zakka, Y. Tassa, and MuJoCo Menagerie contributors, "MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo," 2022. https://github.com/google-deepmind/mujoco_menagerie

[3] K. Zakka, "Mink: Python inverse kinematics based on MuJoCo," software. https://github.com/kevinzakka/mink

[4] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Trans. Syst., Man, Cybern.*, vol. 16, no. 1, pp. 93–101, 1986. https://doi.org/10.1109/TSMC.1986.289285

[5] Y. Nakamura and H. Hanafusa, "Inverse kinematic solutions with singularity robustness for robot manipulator control," *J. Dyn. Syst., Meas., Control*, vol. 108, no. 3, pp. 163–171, 1986. https://doi.org/10.1115/1.3143764

[6] S. R. Buss, "Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods," Tech. rep., UC San Diego, 2004. https://www.math.ucsd.edu/~sbuss/ResearchWeb/ikmethods/iksurvey.pdf

[7] S. Caron *et al.*, "qpsolvers: Quadratic programming solvers in Python," software. https://github.com/qpsolvers/qpsolvers

[8] D. Arnström, A. Bemporad, and D. Axehill, "A dual active-set solver for embedded quadratic programming using recursive LDL$^\top$ updates," *IEEE Trans. Autom. Control*, vol. 67, no. 8, pp. 4362–4369, 2022. https://doi.org/10.1109/TAC.2022.3176430

[9] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927. https://doi.org/10.1080/01621459.1927.10502953

[10] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. https://doi.org/10.1007/BF02295996